

Relatório de Auditoria de APIs, Segurança (OWASP API Top 10) e Performance

Semgrep CLI Visualizer & Executive Dashboard

Projeto Auditado:	Semgrep CLI Visualizer (SPA Client-Side + Nginx + WebMCP)
Domínio de Produção:	https://semgrep.brunoizidorio.com.br
Data da Auditoria:	31 de Agosto de 2026
Auditor Responsável:	Principal API Engineer & AppSec Specialist (Antigravity v2.0)
Metodologia Aplicada:	OWASP API Security Top 10 (2023), STRIDE Threat Modeling, RFC 8288 / 8414 / 9727 / 9728
Classificação:	Confidencial — Uso Interno de Engenharia

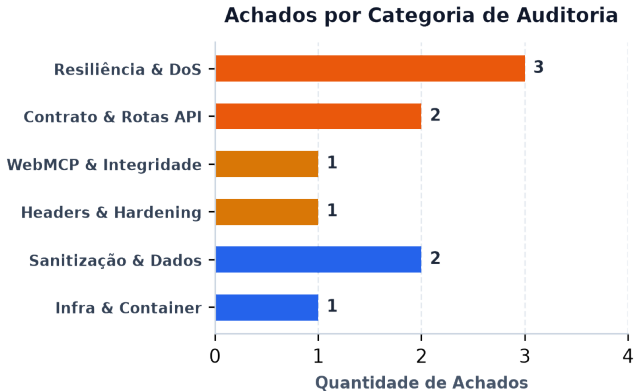
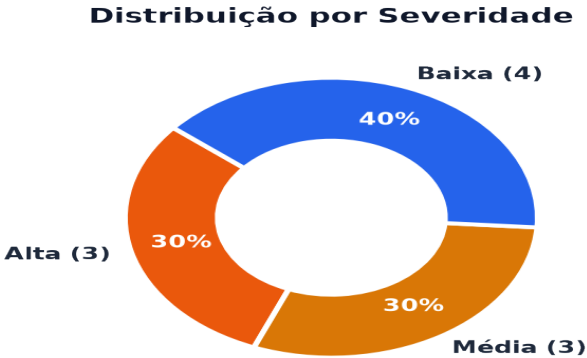
NOTA METODOLÓGICA & ESCOPO DA ARQUITETURA

O **Semgrep CLI Visualizer** opera sob o paradigma **100% Client-Side / Zero-Persistence**, onde todo o processamento de parsing de relatórios SAST, validação via esquemas Zod e cálculo de scores executivos ocorre exclusivamente na memória RAM local do navegador do usuário.

Dada a ausência de um backend tradicional de banco de dados e APIs REST CRUD, a auditoria cobriu com rigor extremo a **superfície de interfaces e rotas expostas**: (1) Servidor Nginx de produção e conformidade de rotas `/.well-known/` de descoberta de IA (RFC 9727, RFC 8414, RFC 9728, SEP-1649); (2) Ferramentas WebMCP in-browser (`navigator.modelContext.provideContext`); (3) Resiliência e mitigação de DoS no upload/drag-and-drop de arquivos JSON pesados; (4) Cabeçalhos de segurança HTTP, isolamento de origem (COOP/COEP/CORP) e políticas de CSP; (5) Service Worker e integridade do cache PWA; (6) Hardening do contêiner Docker multi-stage (UID 101 não-root).

1. Resumo Executivo & Métricas Globais

CRÍTICA 0 0% do total	ALTA 3 30% do total	MÉDIA 3 30% do total	BAIXA 4 40% do total	QUICK WINS 7 70% dos itens
---	---	--	--	--



2. Diagnóstico de Postura: Pontos Fortes vs. Pontos Fracos

■ PONTOS FORTES IDENTIFICADOS • Zero-Persistence Garantida: Nenhum dado sensível de varredura ou código é persistido em disco ou enviado à nuvem. • Validação de Schema Zod Estrita: Bloqueio de injeções de payloads JSON mal formatados no parsing. • Isolamento de Origem Robusto: Implementação de COOP, COEP, CORP e CSP sem 'unsafe-inline' em scripts. • Container Hardened: Execução com usuário sem privilégios (UID 101) na porta 8080.	■ PRINCIPAIS VETORES DE RISCO (PONTOS FRACOS) • Quebra de Contrato em Rotas /well-known/: Nginx entrega SPA HTML com Content-Type: application/json. • Risco de OOM DoS no Dropzone: Falta de validação prévia de file.size antes de FileReader.readAsText(). • Falta de Rate Limiting no Nginx: Ausência de zonas limit_req contra inundações DoS. • Mutação de Estado WebMCP sem Consentimento: Agentes de IA podem sobrescrever a análise do usuário silenciosamente.
--	---

3. Matriz de Priorização e Oportunidades de Quick Wins

A tabela abaixo consolida todos os **10 achados de segurança e resiliência** mapeados no repositório, ordenados pelo índice de criticidade e relação impacto/esforço. Itens marcados como **QUICK WIN** representam correções de alto impacto com tempo de implementação ≤ 30 minutos.

ID	Arquivo / Ponto	Categoria	Severidade	Esforço	Quick Win?
#1	nginx.conf:80-106	Contrato de API (Discovery)	ALTA	Baixo (15 min)	SIM
#2	FileDropzone.tsx:39	Resiliência (Client DoS / OOM)	ALTA	Baixo (10 min)	SIM
#3	nginx.conf:1-174	Resiliência (Rate Limit)	ALTA	Médio (30 min)	NÃO
#4	useSemgrepStore.ts:35	Resiliência (Timeouts)	MÉDIA	Baixo (15 min)	SIM
#5	webMcp.ts:53	WebMCP (Integridade Estado)	MÉDIA	Médio (1 hr)	NÃO
#6	nginx.conf:14-23	Headers HTTP (HSTS/Tokens)	MÉDIA	Baixo (10 min)	SIM
#7	defectdojo.adapter.ts:40	Normalização de Contrato	BAIXA	Baixo (10 min)	SIM
#8	public/sw.js:75	Conformidade HTTP (SW)	BAIXA	Baixo (5 min)	SIM
#9	docker-compose.yml:1-18	Hardening de Container	BAIXA	Baixo (10 min)	SIM
#10	sanitizer.service.ts:5	Integridade de Snippets	BAIXA	Baixo (15 min)	SIM

4. Detalhamento Técnico dos Achados por Categoria

Achado #1: Retorno de HTML como application/json em Rotas /.well-known/ Inexistentes	SEVERIDADE: ALTA Esforço: BAIXO (15 min)
Arquivo/Linha: frontend/nginx.conf:80-106 e frontend/public/ Categoria: OWASP API8:2023 - Security Misconfiguration / API Contract Violation Tag: [QUICK WIN] Problema & Risco: O Nginx possui blocos configurados para /.well-known/oauth-protected-resource e /.well-known/oauth-authorization-server com default_type application/json;, porém com fallback try_files ... /index.html;. Como os arquivos estáticos não existem em frontend/public/.well-known/, o servidor entrega o código HTML do SPA com cabeçalho Content-Type: application/json; charset=utf-8. Agentes de IA e clientes HTTP quebram com SyntaxError na desserialização de JSON. Além disso, endpoints anunciados no header Link (/./well-known/api-catalog, /.well-known/agent-skills/index.json, /.well-known/mcp/server-card.json) retornam 404. Correção Recomendada: 1. Criar os arquivos JSON físicos no diretório frontend/public/.well-known/ com seus respectivos schemas (RFC 9727 linkset+json, SEP-1649 MCP card, Agent Skills index). 2. Remover o fallback /index.html das diretivas de API/JSON no nginx.conf, retornando 404 explícito caso o arquivo não exista.	
Achado #2: Ausência de Validação de file.size no Dropzone Antes de Leitura (OOM Crash)	SEVERIDADE: ALTA Esforço: BAIXO (10 min)
Arquivo/Linha: frontend/src/components/common/FileDropzone.tsx:39-57 Categoria: OWASP API4:2023 - Unrestricted Resource Consumption (Client-Side DoS) Tag: [QUICK WIN] Problema & Risco: A função readFile valida a extensão .json e o MIME type, mas não valida file.size antes de chamar reader.readAsText(file). O limite de 50MB é validado tardiamente dentro de loadJson() no Zustand. Se um usuário arrastar um arquivo de 1GB+, o FileReader tenta alocar o buffer completo na memória RAM do navegador, causando congelamento da UI e travamento por Out Of Memory (OOM). Correção Recomendada: Inserir a verificação defensiva de tamanho logo no início de readFile(file: File): <pre>if (file.size > 50 * 1024 * 1024) { alert('O arquivo selecionado excede o limite máximo de 50MB.'); return; }</pre>	
Achado #3: Ausência de Módulo de Rate Limiting (limit_req) e client_max_body_size no Nginx	SEVERIDADE: ALTA Esforço: MÉDIO (30 min)
Arquivo/Linha: frontend/nginx.conf:1-174 Categoria: OWASP API4:2023 - Unrestricted Resource Consumption (Rate Limiting) Tag: [Planejado] Problema & Risco: O Nginx de produção não define limites de taxa de requisições por IP (limit_req_zone / limit_req) nem restrição explícita de client_max_body_size. Endpoints que servem amostras estáticas pesadas (/samples/semgrep-sample-report.json) ou arquivos de descoberta podem ser alvos de flooding automatizado, consumindo descritores de rede e I/O da VPS. Correção Recomendada: No nginx.conf, declarar na seção http/server: <pre>limit_req_zone \$binary_remote_addr zone=req_limit:10m rate=30r/s; client_max_body_size 50M; E aplicar limit_req zone=req_limit burst=20 nodelay; nos blocos de location.</pre>	
Achado #4: Requisição fetch() sem Timeout / AbortController no Carregamento de Exemplos	SEVERIDADE: MÉDIA Esforço: BAIXO (15 min)

Arquivo/Linha: frontend/src/store/useSemgrepStore.ts:32-43
Categoria: Resiliência & Timeouts em Chamadas Downstream
Tag: [QUICK WIN]

Problema & Risco: A função loadSample() executa fetch('/samples/semgrep-sample-report.json') sem AbortController ou AbortSignal.timeout(). Se a rede oscilar ou o gateway travar, o estado do Zustand permanece indefinidamente bloqueado em isLoading: true sem possibilidade de recuperação pelo usuário.

Correção Recomendada:

Adicionar AbortSignal com timeout de 8 segundos:
`const res = await fetch('/samples/semgrep-sample-report.json', {
 signal: AbortSignal.timeout(8000)
});`

Achado #5: Mutação Direta de Estado Global da Aplicação sem Confirmação do Usuário via WebMCP

SEVERIDADE: MÉDIA | Esforço: MÉDIO (1 hr)

Arquivo/Linha: frontend/src/utills/webMcp.ts:42-54
Categoria: OWASP API5:2023 - Broken Function Level Authorization / WebMCP Integrity
Tag: [Planejado]

Problema & Risco: A ferramenta analyze_semgrep_report exposta para agentes via navigator.modelContext.provideContext() executa useSemgrepStore.setState({ report ... }) diretamente. Scripts ou agentes externos podem sobrescrever a tela de análise do usuário ativo sem qualquer confirmação visual ou diálogo de consentimento.

Correção Recomendada:

Separar a execução do WebMCP em duas modalidades: (a) Retorno puro de métricas estruturadas para o agente sem mutação de tela; (b) Emissão de evento customizado com notificação Toast para que o operador decida se deseja carregar o relatório no dashboard.

Achado #6: Ausência de Cabeçalho HSTS, Ocultação de Versão e Inconsistência no server_name

SEVERIDADE: MÉDIA | Esforço: BAIXO (10 min)

Arquivo/Linha: frontend/nginx.conf:2-23
Categoria: OWASP API8:2023 - Security Misconfiguration / HTTP Headers
Tag: [QUICK WIN]

Problema & Risco: O Nginx não emite o cabeçalho Strict-Transport-Security (HSTS), não desabilita a versão via server_tokens off;, e inclui o token default_server dentro do parâmetro server_name na linha 3 (server_name semgrep.brunoizidorio.com.br default_server;), o que é sintaticamente incorreto.

Correção Recomendada:

No nginx.conf:
1. Alterar para: listen 8080 default_server; server_name semgrep.brunoizidorio.com.br _;
2. Adicionar: server_tokens off;
3. Adicionar: add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;

Achado #7: Tratamento Incorreto de Contrabarras Windows (\) na Extração de Hotspots

SEVERIDADE: BAIXA | Esforço: BAIXO (10 min)

Arquivo/Linha: frontend/src/services/defectdojo.adapter.ts:39-43
Categoria: Normalização de Contratos & Resiliência de Dados
Tag: [QUICK WIN]

Problema & Risco: A função getParentDirectory executa split('/') estrito. Relatórios gerados em ambiente Windows (com caminhos como src\controllers\user.ts) não têm suas pastas extraídas corretamente, sendo todos agrupados sob a pasta 'Raiz do Projeto'.

Correção Recomendada:

Normalizar o caminho antes da divisão:
`function getParentDirectory(filePath: string): string {
 const normalized = filePath.replace(/\\/g, '/');
 const parts = normalized.split('/');
 if (parts.length <= 1) return 'Raiz do Projeto';
 return parts.slice(0, Math.min(2, parts.length - 1)).join('/');
}`

Achado #8: Código HTTP Não Padronizado (488) em Respostas de Falha Offline no Service Worker

SEVERIDADE: BAIXA | Esforço: BAIXO (5 min)

Arquivo/Linha: frontend/public/sw.js:75-79

Categoria: OWASP API8:2023 - HTTP Standards Compliance

Tag: [QUICK WIN]

Problema & Risco: O Service Worker retorna status: 488 quando requisições não cacheadas falham offline. O status 488 não existe no padrão IANA/RFC 9110, causando comportamento anômalo em clientes e ferramentas de telemetria.

Correção Recomendada:

Substituir status 488 pelo código padronizado 503 (Service Unavailable) ou 504 (Gateway Timeout).

Achado #9: Ausência de Limites de Recursos e Filesystem Read-Only no Docker Compose

SEVERIDADE: BAIXA | Esforço: **BAIXO (10 min)**

Arquivo/Linha: docker-compose.yml:1-18

Categoria: Hardening de Infraestrutura & Container Security

Tag: [QUICK WIN]

Problema & Risco: O compose de produção não restringe limites de memória/CPU para o container, permitindo que ataques de DoS saturem os recursos do nó hospedeiro, além de não utilizar read_only: true e no-new-privileges: true.

Correção Recomendada:

Adicionar no docker-compose.yml:

```
security_opt:  
  - no-new-privileges:true  
read_only: true  
tmpfs:  
  - /tmp  
  - /var/run  
  - /var/cache/nginx  
deploy:  
  resources:  
    limits:  
      cpus: '0.50'  
      memory: 256M
```

Achado #10: Mutilação de Tags HTML/XML em Snippets de Código via DOMPurify Destrutivo

SEVERIDADE: BAIXA | Esforço: **BAIXO (15 min)**

Arquivo/Linha: frontend/src/services/sanitizer.service.ts:5 & CodeViewerModal.tsx:181

Categoria: Data Integrity & UI Rendering Defense

Tag: [QUICK WIN]

Problema & Risco: A função sanitizeText utiliza DOMPurify com ALLOWED_TAGS: []. Quando o usuário visualiza snippets de código que contêm tags HTML/XML legítimas (ex: ,) ou tipos genéricos TypeScript (), o DOMPurify remove as tags completamente, desfigurando o código inspecionado. O React já realiza escape seguro nativo ao renderizar strings.

Correção Recomendada:

Permitir renderização fiel de strings no bloco de código utilizando o escape nativo do React JSX, reservando DOMPurify estrito para injeções em contextos HTML ricos.

5. Recomendações Priorizadas de Remediação (Roadmap)

Fase / Prioridade	Ações de Remediação	Achados Relacionados	Prazo Sugerido
P1 - Imediata (Quick Wins Críticos)	1. Criar arquivos estáticos em frontend/public/.well-known/ para sanar a quebra de contrato de API. 2. Inserir validação de file.size <= 50MB no FileDropzone.tsx para prevenir travamento OOM do navegador.	Achados #1, #2	Sprint Atual (< 24h)
P2 - Curto Prazo (Resiliência & Hardening)	1. Configurar limit_req e client_max_body_size no Nginx. 2. Adicionar HSTS, server_tokens off; e ajustar server_name. 3. Configurar timeout com AbortSignal.timeout(8000) na store Zustand.	Achados #3, #4, #6	Próxima Sprint (1 semana)
P3 - Médio Prazo (WebMCP & Integridade)	1. Refatorar ferramenta WebMCP para exigir consentimento ou operar em modo headless sem mutação destrutiva da UI. 2. Aplicar restrições de CPU/RAM (cgroups) e read_only: true no Docker Compose.	Achados #5, #9	Próximas 2 semanas
P4 - Melhoria Contínua	1. Normalizar separadores de caminho Windows (\) no adapter DefectDojo. 2. Corrigir código de status HTTP do Service Worker (503 em vez de 488). 3. Preservar tags HTML em snippets de código.	Achados #7, #8, #10	Backlog Técnico

6. Issues Prontas para o GitHub (Templates Completos)

Os blocos delimitados a seguir contêm o texto integral em Markdown formatado para abertura direta de Issues no GitHub. Cada item possui critérios de aceite verificáveis e orientações de implementação.

```
--- ISSUE 1 ---
## [API/Segurança] Correção de Retorno HTML em Rotas /.well-known/ e Criação de Catálogos de Descoberta (RFC 9727 / RFC 8414)

**Labels:** `api`, `security`, `high-priority`, `bug`

### Descrição do Problema
As rotas de descoberta e metadados OAuth configuradas no Nginx (`/.well-known/oauth-protected-resource` e `/.well-known/oauth-authorization-server`) retornam o código HTML do SPA (`index.html`) com cabeçalho `Content-Type: application/json; charset=utf-8` devido ao fallback `try_files` para `/index.html` e à ausência dos arquivos físicos em `frontend/public/.well-known/`. Agentes de IA e ferramentas automatizadas falham ao tentar decodificar a resposta JSON.

### Evidência
- `frontend/nginx.conf:80-106`: Diretiva `try_files /.well-known/oauth-protected-resource /index.html;` com `default_type application/json;`.
- `frontend/public/`: Diretório `.well-known/` não existe.

### Impacto
Quebra severa de contrato de API (Content-Type Mismatch) e impossibilidade de descoberta automatizada de capacidades por agentes compatíveis com RFC 9727 e RFC 8414.

### Sugestão de Correção
1. Criar o diretório `frontend/public/.well-known/` contendo:
  - `api-catalog` (`application/linkset+json`)
  - `oauth-authorization-server` (`application/json`)
  - `oauth-protected-resource` (`application/json`)
  - `mcp/server-card.json` (`application/json`)
  - `agent-skills/index.json` (`application/json`)
2. Ajustar `nginx.conf` para servir os arquivos estáticos diretamente e retornar 404 caso não existam, sem fallback para `index.html`.

### Critérios de Aceite
- [ ] `curl -H "Accept: application/json" https://semgrep.brunoizidorio.com.br/.well-known/oauth-authorization-server` retorna JSON válido.
- [ ] `curl -H "Accept: application/linkset+json" https://semgrep.brunoizidorio.com.br/.well-known/api-catalog` retorna 200 OK com linkset JSON.
- [ ] Teste automatizado em `tests/agentDiscovery.test.ts` validando o conteúdo de todos os endpoints `.well-known/`.

--- FIM ISSUE 1 ---
```



```
--- ISSUE 2 ---
## [API/Segurança] Validação Defensiva de file.size no Dropzone para Mitigar Client-Side DoS (OOM Crash)

**Labels:** `security`, `performance`, `high-priority`, `quick-win`

#### Descrição do Problema
O componente `FileDropzone.tsx` valida a extensão `.json` e o MIME type, mas não verifica o tamanho do arquivo (`file.size`) antes de invocar `FileReader.readAsText()`. A restrição de 50MB é aplicada apenas dentro da store Zustand. Arquivos gigantes (>500MB) provocam estouro de memória e congelamento da aba antes da validação.

#### Evidência
- `frontend/src/components/common/FileDropzone.tsx:39-57`: Chamada a `reader.readAsText(file)` sem verificação prévia de `file.size`.

#### Impacto
Negação de serviço no cliente (Client-Side Denial of Service / Browser Crash) em caso de seleção acidental de arquivos massivos.

#### Sugestão de Correção
Adicionar verificação no início de `readFile`:
```typescript
if (file.size > 50 * 1024 * 1024) {
 alert(t('jsonSizeErrorAlert') || 'O arquivo excede o limite máximo de 50MB.');
```

```
--- ISSUE 3 ---
[API/Performance] Implementação de Rate Limiting e client_max_body_size no Nginx

Labels: `api`, `performance`, `infrastructure`, `high-priority`

Descrição do Problema
O arquivo de configuração do Nginx não possui controle de taxa de requisições (`limit_req_zone` / `limit_req`) nem definição de `client_max_body_size`, expondo os endpoints estáticos e relatórios de exemplo a ataques de força bruta e exaustão de descritores de rede.

Evidência
- `frontend/nginx.conf:1-174`: Ausência de diretivas `limit_req_zone` e `client_max_body_size`.

Impacto
Possibilidade de saturação de banda e consumo excessivo de CPU/RAM no servidor VPS por requisições concorrentes massivas.

Sugestão de Correção
Adicionar no `nginx.conf`:
```nginx
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=30r/s;
client_max_body_size 50M;

# Nos blocos de location:
limit_req zone=api_limit burst=20 nodelay;
```

Critérios de Aceite
- [] Disparo de mais de 50 requisições simultâneas em `/samples/semgrep-sample-report.json` resulta em HTTP 429 / 503 com `limit_req`.
- [] Configuração validada via `nginx -t` e testes em `tests/nginx.config.test.ts`.
--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---
[API/Performance] Configuração de Timeout com AbortController no useSemgrepStore

Labels: `api`, `performance`, `resilience`, `medium-priority`, `quick-win`

Descrição do Problema
O método `loadSample` na store Zustand executa `fetch('/samples/semgrep-sample-report.json')` sem `AbortController` ou `AbortSignal.timeout()`. Conexões instáveis podem deixar a aplicação bloqueada permanentemente no estado `isLoading: true`.
```

### Evidência

```
- `frontend/src/store/useSemgrepStore.ts:35`: `const res = await fetch('/samples/semgrep-sample-report.json');` s em `signal`.
```

### Impacto

Degradação de usabilidade e falta de resiliência em chamadas HTTP downstream.

### Sugestão de Correção

```
``typescript
const res = await fetch('/samples/semgrep-sample-report.json', {
 signal: AbortSignal.timeout(8000)
});
``
```

### Critérios de Aceite

- [ ] Requisição cancelada automaticamente após 8 segundos em caso de timeout de rede.
- [ ] Teste unitário simulando erro de timeout e retorno do estado `isLoading: false`.

```
--- FIM ISSUE 4 ---
```

```
--- ISSUE 5 ---
[API/Segurança] Adição de Cabeçalho HSTS, Ocultação de Versão e Correção do server_name no Nginx

Labels: `security`, `infrastructure`, `medium-priority`, `quick-win`

Descrição do Problema
Ausência do cabeçalho `Strict-Transport-Security` (HSTS), ausência de `server_tokens off;` para ocultar o banner do Nginx, e uso de `default_server` no parâmetro `server_name`.
```

### Evidência

```
- `frontend/nginx.conf:2-3`: `server_name semgrep.brunoizidorio.com.br default_server;`
- `frontend/nginx.conf:14-23`: Ausência de HSTS e `server_tokens off;`.
```

### Impacto

Vulnerabilidade a ataques de SSL-strip e enumeração de versão do servidor web por atacantes.

### Sugestão de Correção

Ajustar `nginx.conf`:

```
``nginx
listen 8080 default_server;
server_name semgrep.brunoizidorio.com.br _;
server_tokens off;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;
``
```

### Critérios de Aceite

- [ ] Respostas HTTP incluem cabeçalho `Strict-Transport-Security`.
- [ ] Header `Server:` não expõe a versão do Nginx em páginas de erro.

```
--- FIM ISSUE 5 ---
```

```
--- ISSUE 6 ---
[API/Segurança] Isolamento e Notificação de Consentimento para Mutações via WebMCP

Labels: `api`, `security`, `webmcp`, `medium-priority`

Descrição do Problema
A ferramenta WebMCP `analyze_semgrep_report` executa `useSemgrepStore.setState()` diretamente, permitindo que agentes externos alterem a visualização ativa na tela do operador sem notificação ou diálogo de confirmação.

Evidência
- `frontend/src/utils/webMcp.ts:53`: `useSemgrepStore.setState({ report, isLoading: false, error: null });`

Impacto
Perda de integridade da sessão do usuário por automações de terceiros ou agentes descontrolados.

Sugestão de Correção
Separar o retorno dos dados processados (para o agente consumidor) e emitir uma notificação Toast na interface para a autorização do usuário antes de substituir a análise ativa.

Critérios de Aceite
- [] Invocação WebMCP retorna as métricas para o agente sem mutação forçada da tela.
- [] Teste unitário em `tests/agentDiscovery.test.ts` validando o novo fluxo de consentimento.
--- FIM ISSUE 6 ---
```

```
--- ISSUE 7 ---
[API/Qualidade] Normalização de Caminhos de Arquivo Windows no DefectDojo Adapter

Labels: `bug`, `contract`, `low-priority`, `quick-win`

Descrição do Problema
A função `getParentDirectory` realiza `split('/')` estrito, falhando ao extrair a hierarquia de pastas em relatórios gerados no Windows com contrabarras (`\`).

Evidência
- `frontend/src/services/defectdojo.adapter.ts:39-43`: `const parts = filePath.split('/');`

Impacto
Métricas de Top Hotspots distorcidas para usuários em ambiente Windows.

Sugestão de Correção
```typescript
function getParentDirectory(filePath: string): string {
  const normalized = filePath.replace(/\\/g, '/');
  const parts = normalized.split('/');
  if (parts.length <= 1) return 'Raiz do Projeto';
  return parts.slice(0, Math.min(2, parts.length - 1)).join('/');
}
```

Critérios de Aceite
- [] Caminho `src\backend\auth.ts` é agrupado sob `src/backend`.
- [] Teste unitário cobrindo caminhos com separadores Windows e POSIX.
--- FIM ISSUE 7 ---
```

```
--- ISSUE 8 ---
[API/Infra] Hardening de Recursos e Sistema Somente-Leitura no Docker Compose

Labels: `security`, `docker`, `infrastructure`, `low-priority`, `quick-win`

Descrição do Problema
O arquivo `docker-compose.yml` não especifica limites de recursos (`limits`), nem ativa `read_only: true` ou `no-new-privileges: true`.

Evidência
- `docker-compose.yml:1-18`: Ausência de `deploy.resources.limits` e `security_opt`.

Impacto
Risco de exaustão de recursos do host e falta de defesa em profundidade no contêiner.

Sugestão de Correção
Adicionar restrições de cgroups e segurança no serviço `semgrep-visualizer` do compose.

Critérios de Aceite
- [] Contêiner executa com sistema de arquivos somente-leitura (`read_only: true`).
- [] Limites de 256MB RAM e 0.5 CPU aplicados com sucesso.
--- FIM ISSUE 8 ---
```